TP n°2

Requêtes, dépendances fonctionnelles et normalisation

Bouattay Ferdaous

Matière : Bases de données Avancées

Exercice 1 — Requêtes avancees

---

1. Département au budget le plus élevé

```
SELECT dept_name
FROM department
WHERE budget IN (SELECT MAX(budget) FROM department);
```

2. Enseignants gagnant plus que le salaire moyen

```
SELECT name, salary
FROM instructor
WHERE salary > (SELECT AVG(salary) FROM instructor);
```

3. étudiants ayant suivi plus de 2 cours d'un Même enseignant (avec HAVING)

```
SELECT instructor.name, student.name, COUNT(*)
FROM instructor, student, takes, teaches
WHERE instructor.ID = teaches.ID
  AND student.ID = takes.ID
  AND (takes.course_id, takes.sec_id, takes.semester, takes.
     year)
    = (teaches.course_id, teaches.sec_id, teaches.semester,
       teaches.year)
GROUP BY instructor.name, student.name
HAVING COUNT(*) >= 2;
```

4. Même requête sans HAVING

```
SELECT T.tname, T.sname, T.total
FROM (
    SELECT instructor.name AS tname, student.name AS sname,
       COUNT(*) AS total
    FROM instructor, student, takes, teaches
    WHERE instructor.ID = teaches.ID
      AND student.ID = takes.ID
      AND (takes.course_id, takes.sec_id, takes.semester,
         takes.year)
        = (teaches.course_id, teaches.sec_id, teaches.
           semester, teaches.year)
    GROUP BY instructor.name, student.name
) T
WHERE T.total >= 2
ORDER BY T.tname;
```

5. étudiants n'ayant pas suivi de cours avant 2010

```
SELECT ID, name FROM student
MINUS
SELECT student.ID, student.name
FROM student, takes
WHERE takes.ID = student.ID AND takes.year < 2010;
```

On utilise `MINUS` en Oraclé (équivalent de `EXCEPT` en SQL standard).

6. Enseignants dont le nom commence par E

```
SELECT * FROM instructor WHERE name LIKE 'E%';
```

7. Enseignants percevant le quatrieme salaire le plus élevé

```
SELECT name, salary
FROM instructor T1
WHERE 3 = (
    SELECT COUNT(DISTINCT T2.salary)
    FROM instructor T2
    WHERE T2.salary > T1.salary
);
```

La sous-requête compte les salaires strictement supérieurs à celui de T1. Quand ce compte vaut 3, T1 percoit le 4eme salaire le plus élevé.

8. Les trois enseignants avec les salaires les moins élevés (ordre decroissant)

```
SELECT name, salary
FROM instructor T1
WHERE 2 >= (
    SELECT COUNT(DISTINCT T2.salary)
    FROM instructor T2
    WHERE T2.salary < T1.salary
)
ORDER BY salary DESC;
```

9. étudiants ayant suivi un cours en automne 2009 (clause IN)

```
SELECT name
FROM student S
WHERE ('Fall', 2009) IN (
    SELECT semester, year FROM takes WHERE takes.ID = S.ID
);
```

10. Même rêquete avec la clause SOME

```
SELECT name
FROM student S
WHERE ('Fall', 2009) = SOME (
    SELECT semester, year FROM takes WHERE takes.ID = S.ID
);
```

11. Même rêquete avec NATURAL INNER JOIN

```
SELECT DISTINCT student.name
FROM takes NATURAL INNER JOIN student
WHERE takes.semester = 'Fall' AND takes.year = 2009;
```

12. Même rêquete avec la clause EXISTS

```
SELECT name
FROM student
WHERE EXISTS (
    SELECT * FROM takes
    WHERE takes.ID = student.ID
      AND semester = 'Fall'
      AND year = 2009
);
```

13. Paires d'étudiants ayant suivi au moins un cours ensemble

```
SELECT A.name, B.name
FROM (student NATURAL INNER JOIN takes) A,
     (student NATURAL INNER JOIN takes) B
WHERE A.course_id = B.course_id
  AND A.sec_id    = B.sec_id
  AND A.semester  = B.semester
  AND A.year      = B.year
  AND A.ID <> B.ID
  AND A.name < B.name
GROUP BY A.ID, B.ID, A.name, B.name
HAVING COUNT(*) >= 1;
```

14. Nombre d'étudiants par enseignant (seulement ceux ayant enseigne)

```
SELECT instructor.name, COUNT(*)
FROM (takes INNER JOIN teaches
      USING (course_id, sec_id, semester, year))
INNER JOIN instructor ON teaches.ID = instructor.ID
GROUP BY instructor.name, instructor.ID
```

```
ORDER BY COUNT(*) DESC;
```

15. Nombre d'étudiants par enseignant (tous les enseignants, Même sans cours)

```
SELECT instructor.name, COUNT(course_id)
FROM (takes INNER JOIN teaches
      USING (course_id, sec_id, semester, year))
RIGHT OUTER JOIN instructor ON teaches.ID = instructor.ID
GROUP BY instructor.name, instructor.ID
ORDER BY COUNT(course_id) DESC;
```

16. Nombre de grades A attribues par enseignant

```
WITH mytakes AS (
    SELECT ID, course_id, sec_id, semester, year, grade
    FROM takes WHERE grade = 'A'
)
SELECT instructor.name, COUNT(course_id)
FROM (mytakes INNER JOIN teaches
      USING (course_id, sec_id, semester, year))
RIGHT OUTER JOIN instructor ON teaches.ID = instructor.ID
GROUP BY instructor.name, instructor.ID
ORDER BY COUNT(course_id) DESC;
```

17. Paires enseignant-etudiant et nombre de cours suivis ensemble

```
SELECT instructor.name, student.name, COUNT(*)
FROM (instructor NATURAL JOIN teaches)
INNER JOIN (takes NATURAL JOIN student)
USING (course_id, sec_id, semester, year)
GROUP BY instructor.name, student.name;
```

18. Paires enseignant-etudiant avec au moins deux cours suivis ensemble

```
SELECT T.tname, T.sname
FROM (
    SELECT instructor.name AS tname, student.name AS sname,
       COUNT(*) AS total
    FROM (instructor NATURAL JOIN teaches)
    INNER JOIN (takes NATURAL JOIN student)
    USING (course_id, sec_id, semester, year)
    GROUP BY instructor.name, student.name
) T
WHERE T.total >= 2;
```

Exercice 2 — Formes normales

---

1. $R(A, B, C)$ **avec** $F = \{A \to B,\ B \to C\}$

La clé candidate est $\{A\}$ car $A^+ = \{A, B, C\} = R$.

La dependance $B \to C$ viole la BCNF car B n'est pas une super-clé. On decompose :

- $R_1(B, C)$ avec $F_1 = \{B \to C\}$ — en BCNF, clé : $\{B\}$
- $R_2(A, B)$ avec $F_2 = \{A \to B\}$ — en BCNF, clé : $\{A\}$

La relation initiale n'est pas en BCNF. Apres decomposition, on obtient deux relations en BCNF.

2. $R(A, B, C)$ **avec** $F = \{A \to C,\ A \to B\}$

La clé candidate est $\{A\}$ car $A^+ = \{A, B, C\} = R$.

Toutes les dépendances ont A comme determinant, et A est la clé. La relation est donc directement en **BCNF**.

3. $R(A, B, C)$ **avec** $F = \{AB \to C,\ C \to B\}$

Les clés candidates sont $\{A, B\}$ et $\{A, C\}$.

La dependance $C \to B$ viole la BCNF car C n'est pas une super-clé. On decompose :

- $R_1(B, C)$ avec $F_1 = \{C \to B\}$ — en BCNF, clé : $\{C\}$
- $R_2(A, C)$ avec $F_2 = \emptyset$ — en BCNF, clé : $\{A, C\}$

Exercice 3 — dépendances fonctionnelles et BCNF

---

1. Deduction de 16 DF par Armstrong

$R(A, B, C, D, E)$, $F = \{A \to B, C \ ; \ C, D \to E \ ; \ B \to D \ ; \ E \to A\}$

On applique les regles de reflexivite, augmentation et transitivite :

1. $A \to B$ (donne)
2. $A \to C$ (donne)
3. $B \to D$ (donne)
4. $C, D \to E$ (donne)
5. $E \to A$ (donne)
6. $A \to D$ (transitivite : $A \to B$ et $B \to D$)
7. $A \to B, C, D$ (union : $A \to B$, $A \to C$, $A \to D$)
8. $A, C \to E$ (augmentation de $C, D \to E$ + $A \to D$)
9. $A \to E$ (transitivite : $A \to C, D$ et $C, D \to E$)
10. $A \to A$ (reflexivite)
11. $E \to B$ (transitivite : $E \to A$ et $A \to B$)
12. $E \to C$ (transitivite : $E \to A$ et $A \to C$)
13. $E \to D$ (transitivite : $E \to B$ et $B \to D$)
14. $E \to B, C, D$ (union des precedentes)
15. $E \to E$ (reflexivite)
16. $B, C \to E$ (augmentation de $C, D \to E$ et $B \to D$)

2. Fermeture, super-clé et BCNF

$R(A, B, C, D, E, F)$, $F = \{A \to B, C, D \ ; \ B, C \to D, E \ ; \ B \to D \ ; \ D \to A\}$

(a) Fermetures

$B^+ = \{B, D, A, C, E\}$ — on calcule : $B \to D$, puis $D \to A$, puis $A \to B, C, D$, puis $B, C \to D, E$. Donc $B^+ = \{A, B, C, D, E\}$.

$\{A, B\}^+ = \{A, B, C, D, E\}$ par les Mêmes etapes.

(b) $\{A, F\}$ est une super-clé

$\{A, F\}^+ = \{A, B, C, D, E, F\} = R$ car $A \to B, C, D$, puis $B, C \to E$, et F est deja present. Donc $\{A, F\}$ est bien une super-clé.

(c) R est-elle en BCNF ?

Non. La dependance $B \to D$ viole la BCNF car B n'est pas une super-clé ($B^+ \neq R$ puisque $F \notin B^+$). On decompose :

- $R_1(B, D)$ — $B \to D$, clé : $\{B\}$, en BCNF
- $R_2(A, B, C, E, F)$ — on re-verifie les DF restantes

On continue jusqu'a ce que toutes les relations soient en BCNF.

3. Decomposition sans perte et avec perte

(a) $R_1(A, B, C)$ et $R_2(A, D, E)$ — sans perte

$R_1 \cap R_2 = \{A\}$. Si A est une clé de R_1 ou de R_2, la decomposition est sans perte (theoreme de Heath). Ici A determine B, C via les DF de R, donc la jointure naturelle sur A reconstitue R exactement.

(b) $R_1(A, B, C)$ et $R_2(C, D, E)$ — avec perte

$R_1 \cap R_2 = \{C\}$. Si C n'est pas une clé de R_1 ni de R_2, la jointure peut generer des tuples supplementaires qui n'existaient pas dans R d'origine. On peut le montrer par un contre-exemple concret :

Soit R contenant les tuples $(a_1, b_1, c_1, d_1, e_1)$ et $(a_2, b_2, c_1, d_2, e_2)$. R_1 contient (a_1, b_1, c_1) et (a_2, b_2, c_1). R_2 contient (c_1, d_1, e_1) et (c_1, d_2, e_2). La jointure sur C produit 4 tuples au lieu de 2 : perte d'information.

Exercice 4 — Implementation en Python

---

1. Affichage des dépendances fonctionnelles

```
def printDependencies(F: "list of dependencies"):
    for alpha, beta in F:
        print("\t", alpha, " --> ", beta)
```

2. Affichage des relations

```
def printRelations(T: "list of relations"):
    for R in T:
        print("\t", R)
```

3. Calcul de l'ensemble des parties (powerSet)

```
import itertools

def powerSet(inputset: "set"):
    _result = []
    for r in range(1, len(inputset) + 1):
        _result += map(set, itertools.combinations(inputset,
            r))
    return _result
```

4. Fermeture d'un ensemble d'attributs

```
def computeAttributeClosure(F: "list of dependencies", K: "
   set of attributes"):
    K_plus, size = set(K), 0
    while size != len(K_plus):
        size = len(K_plus)
        for alpha, beta in F:
            if alpha.issubset(K_plus):
                K_plus.update(beta)
    return K_plus
```

On part de K et on applique itérativement les dépendances fonctionnelles jusqu'a stabilisation. La fermeture K^+ contient tous les attributs determines par K.

5. Fermeture de F

```
def computeDependenciesClosure(F: "list of dependencies"):
    R = set()
```

```
    for alpha, beta in F:
        R.update(alpha | beta)

    F_plus = []
    for K in powerSet(R):
        for beta in powerSet(computeAttributeClosure(F, K)):
            F_plus.append([K, beta])
    return F_plus
```

6. Verification d'une dependance fonctionnelle

```
def isDependency(F: "list of dependencies",
                 alpha: "set of attributes",
                 beta: "set of attributes"):
    return beta.issubset(computeAttributeClosure(F, alpha))
```

7. Verification d'une super-clé

```
def isSuperKey(F: "list of dependencies",
               R: "set defining a relation",
               K: "set of attributes"):
    return R.issubset(computeAttributeClosure(F, K))
```

8. Verification d'une clé candidate

```
def isCandidateKey(F: "list of dependencies",
                   R: "set defining a relation",
                   K: "set of attributes"):
    if not isSuperKey(F, R, K):
        return False
    for A in K:
        K1 = set(K)
        K1.discard(A)
        if isSuperKey(F, R, K1):
            return False
    return True
```

Une clé candidate est une super-clé minimale : on verifie qu'aucun attribut ne peut etre retire tout en conservant la propriete de super-clé.

9. Liste de toutes les clés candidates

```
def computeAllCandidateKeys(F: "list of dependencies",
                            R: "set defining a relation"):
    result = []
```

```
    for K in powerSet(R):
        if isCandidateKey(F, R, K):
            result.append(K)
    return result
```

10. Liste de toutes les super-clés

```
def computeAllSuperKeys(F: "list of dependencies",
                        R: "set defining a relation"):
    result = []
    for K in powerSet(R):
        if isSuperKey(F, R, K):
            result.append(K)
    return result
```

11. Calcul d'une clé candidate

```
def computeOneCandidateKey(F: "list of dependencies",
                           R: "set defining a relation"):
    K = set(R)
    while not isCandidateKey(F, R, K):
        for A in K:
            if isSuperKey(F, R, K.difference({A})):
                K.remove(A)
                break
    return K
```

On part de la relation entiere et on retire itérativement les attributs superflus jusqu'a obtenir une clé candidate minimale.

12. Verification BCNF pour une relation

```
def isBCNFRelation(F: "list of dependencies",
                   R: "set defining a relation"):
    for K in powerSet(R):
        K_plus = computeAttributeClosure(F, K)
        Y = K_plus.difference(K)
        if not R.issubset(K_plus) and not Y.isdisjoint(R):
            return False, [K, Y & R]
    return True, [{}, {}]
```

13. Verification BCNF pour un schema

```
def isBCNFRelations(F: "list of dependencies",
                    T: "list of relations"):
```

```
    for R in T:
        result, witness = isBCNFRelation(F, R)
        if result == False:
            return False, R
    return True, {}
```

14. Algorithme de decomposition en BCNF

```
def computeBCNFDecomposition(F: "list of dependencies",
                             T: "list of relations"):
    OUT, size = list(T), 0
    while size != len(OUT):
        size = len(OUT)
        for R in OUT:
            is_bcnf, [alpha, beta] = isBCNFRelation(F, R)
            if not is_bcnf:
                if alpha | beta not in OUT:
                    OUT.append(alpha | beta)
                if R.difference(beta) not in OUT:
                    OUT.append(R.difference(beta))
                OUT.remove(R)
                break
    return OUT
```

L'algorithme décompose itérativement chaque relation qui viole la BCNF en deux sous-relations, jusqu'à ce que toutes soient en BCNF. A chaque étape, la relation violatrice R est remplacée par $R_1 = \alpha \cup \beta$ et $R_2 = R \setminus \beta$.